Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ДОДАТКОВЕ ЗАВДАННЯ № 2

«Операції над матрицями»

з курсу «Основи програмування»

Виконав:
Студент гр. ФеП-13
Шмирко В. М.

Перевірив:
ас. Васюта В. М.

Львів 2026

1. Математичні основи.

Матриця — прямокутна таблиця чисел (елементів), розташованих у рядках і стовпцях. Матрицю розміром m × n (m рядків, n стовпців) позначають як $A = (a_{ij})$, де i = 1, …, m; j = 1, …, n. Нижче наведено основні операції над матрицями, реалізовані в програмі.

1.1. Транспонування матриці.

Транспонована матриця A^T утворюється шляхом заміни рядків матриці A на стовпці (і навпаки). Якщо A має розмір m × n, то A^T має розмір n × m, і $(A^T)_{ij} = a_{ji}$. Транспонування зберігає лінійні властивості і є інволюцією: $(A^T)^T = A$.

1.2. Множення матриці на скаляр.

Добутком матриці $A = (a_{ij})$ на скаляр (число) λ є матриця λA того ж розміру, елементи якої: $(\lambda A)_{ij} = \lambda \cdot a_{ij}$.

1.3. Додавання та віднімання матриць.

Додавання (віднімання) двох матриць A і B можливе лише тоді, коли вони мають однаковий розмір m × n. Результатом є матриця C того ж розміру:

$$c_{ij} = a_{ij} + b_{ij} \text{ (додавання)}, \quad c_{ij} = a_{ij} - b_{ij} \text{ (віднімання)}.$$

1.4. Множення матриць.

Добутком матриці A розміру m × p та матриці B розміру p × n є матриця C розміру m × n, елементи якої обчислюються за формулою:

$$c_{ij} = \Sigma_{k=1}^{p} a_{ik} \cdot b_{kj}.$$

Необхідною умовою множення є рівність кількості стовпців першої матриці кількості рядків другої. Множення матриць, загалом, некомутативне: AB ≠ BA. Усі наведені операції є базовими в лінійній алгебрі та широко застосовуються у чисельних методах, комп’ютерній графіці, розв’язуванні систем лінійних рівнянь тощо.

2. Алгоритм програми.

У вигляді псевдокоду:

1. Початок.
2. Вивести меню операцій над матрицями (транспонування, множення на скаляр, додавання, віднімання, множення).
3. Зчитати вибір користувача (змінна temp).
4. Залежно від вибору (switch):
4.1. Ввести розміри матриці (матриць) та виділити пам’ять (allocmatrix).
4.2. Перевірити коректність розмірів (для додавання/віднімання — однаковість; для множення — узгодженість).
4.3. Ввести елементи матриці (матриць) за допомогою input_matrix / f_input.
4.4. Виконати обрану операцію (transponate / mult_by_scalar / add_matrixes / minus_matrixes / mult_by_matrix).
4.5. Вивести результуючу матрицю (print_matrix).
5. Звільнити виділену пам’ять (freematr).

6. Кінець.

У вигляді блок-схеми:

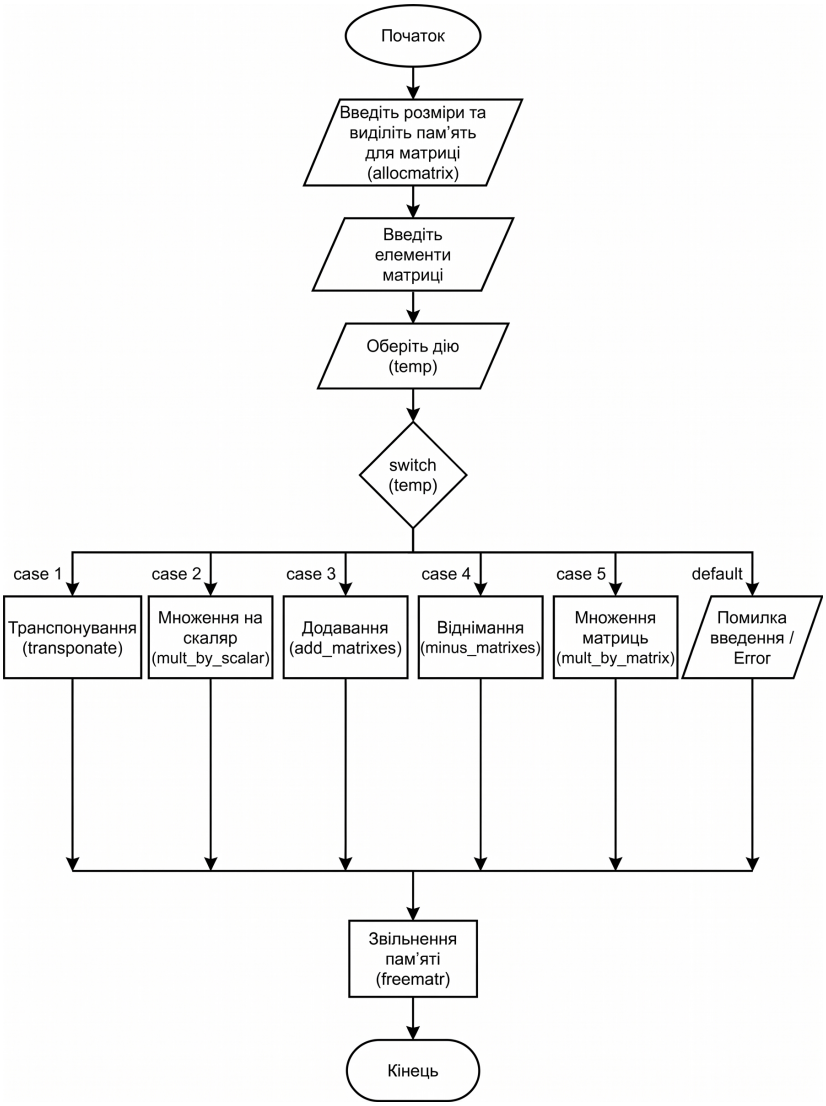


Рис. 0. Блок-схема алгоритму програми

3. Опис програми.

Програма реалізує основні операції над матрицями дійсних чисел типу double: транспонування, множення матриці на скаляр, додавання, віднімання та множення двох матриць. Робота починається з виведення текстового меню, після чого користувач обирає потрібну операцію шляхом введення цілого числа від 1 до 5. Далі, залежно від вибору, програма запитує розміри та елементи матриць, виконує обчислення і виводить результат. Усі матриці створюються динамічно, що дозволяє працювати з матрицями довільного (у межах доступної пам'яті) розміру.

Для безпечного введення даних використовуються дві допоміжні функції. Функція i_input організовує ввід цілого числа. Вона намагається зчитати значення за допомогою scanf("%d", …). Якщо введення невдале (користувач ввів нечислове значення), функція очищає вхідний буфер (while (getchar() != '\n')), виводить повідомлення «ПОМИЛКАААААА!!!введіть число ще раз :» і рекурсивно викликає саму себе, поки не буде отримано коректне ціле число. Аналогічно

працює функція f_input для дійсних чисел типу double (формат "%lf").

Динамічне виділення пам’яті для матриць виконує функція allocmatrix(int r, int c). Вона спочатку виділяє масив із r вказівників на рядки (double**), а потім для кожного рядка виділяє масив із c елементів типу double. Повертається вказівник на початок матриці. Звільнення пам’яті виконує функція freematr(double **matr, int r): спочатку звільняються усі рядки, після чого — сам масив вказівників. Такий підхід запобігає витокам пам’яті.

Ввід елементів матриці здійснюється функцією input_matrix(double **matr, int r, int c). Вона послідовно (рядок за рядком, стовпець за стовпцем) викликає f_input для кожного елемента. Вивід результату виконує функція print_matrix(double **matr, int r, int c): спочатку виводиться заголовок «Результат:», після чого матриця друкується з форматуванням «%8.2lf» (вісім позицій під число, два знаки після коми), що забезпечує акуратне вирівнювання стовпців.

Основні обчислювальні функції реалізовані наступним чином.

Функція transponate(double **matr, int r, int c) створює нову матрицю розміру c × r за допомогою allocmatrix і заповнює її елементи за правилом res[j][i] = matr[i][j]. Повертається вказівник на транспоновану матрицю.

Функція mult_by_scalar(double **matr, int r, int c, double x) створює матрицю того ж розміру r × c і обчислює кожен елемент як res[i][j] = matr[i][j] * x. Повертає вказівник на результат.

Функції add_matrixes та minus_matrixes (параметри: дві матриці A і B та їх розміри r, c) створюють результуючу матрицю того ж розміру і поелементно обчислюють суму або різницю: res[i][j] = A[i][j] ± B[i][j]. Перед викликом цих функцій у main перевіряється, що розміри обох матриць збігаються; у разі незбігу виводиться повідомлення про помилку і програма коректно завершується після звільнення вже виділеної пам’яті.

Функція mult_by_matrix(double **A, int ra, int ca, double **B, int rb, int cb) реалізує класичне матричне множення. Спочатку виділяється пам’ять під матрицю розміру ra × cb. Кожен елемент res[i][j] ініціалізується нулем, після чого в внутрішньому циклі по k накопичується сума A[i][k] * B[k][j]. Перед викликом перевіряється умова ca == rb (кількість стовпців першої матриці дорівнює кількості рядків другої); при порушенні умови виводиться відповідне повідомлення про помилку.

У функції main після виконання обраної операції та виводу результату викликається freematr для усіх створених матриць (вхідних і результуючої). Це гарантує відсутність витоків пам’яті навіть при багаторазовому запуску програми. У гілці default (некоректний вибір) виводиться повідомлення «Такої функції не існує!» і програма завершується.

Таким чином, програма поєднує зручний інтерактивний інтерфейс, надійну обробку помилок вводу, динамічне управління пам’яттю та коректну реалізацію усіх п’яти базових операцій над матрицями.

4. Приклади роботи програми.

4.1. Транспонування матриці 3×3.

```
Оберіть дію:
1. Транспонування матриці
2. Множення матриці на скаляр
3. Додавання матриць
4. Віднімання матриць
5. Множення матриць
1
Введіть кількість рядків та стовпчиків матриці (через пробіл):
3 3
Введіть саму матрицю:
1 2 3
4 5 6
7 8 9
Результат:
    1.00     4.00     7.00
    2.00     5.00     8.00
    3.00     6.00     9.00
```

Рис. 1. Транспонування матриці

4.2. Множення матриці на скаляр.

```
Оберіть дію:
1. Транспонування матриці
2. Множення матриці на скаляр
3. Додавання матриць
4. Віднімання матриць
5. Множення матриць
2
Введіть кількість рядків та стовпчиків матриці (через пробіл):
2 3
Введіть саму матрицю:
3 5 8.3
2 2 1.5
Введіть скаляр: 3
Результат:
    9.00    15.00    24.90
    6.00     6.00     4.50
```

Рис. 2. Множення матриці 2×3 на скаляр 3

4.3. Додавання матриць 4×4.

```
Оберіть дію:
1. Транспонування матриці
2. Множення матриці на скаляр
3. Додавання матриць
4. Віднімання матриць
5. Множення матриць
3
Введіть кількість рядків та стовпчиків матриці номер 1 (через пробіл):
4 4
Введіть саму матрицю номер 1:
1 5 6 2
33 4 2 1
8 3 4 1
1 6 6 1
Введіть кількість рядків та стовпчиків матриці номер 2 (через пробіл):
4 4
Введіть саму матрицю номер 2:
6 7 5 2
2 0 0 8
3 44 2 1.5
3.5 2 2 1
Результат:
    7.00    12.00    11.00     4.00
   35.00     4.00     2.00     9.00
   11.00    47.00     6.00     2.50
    4.50     8.00     8.00     2.00
```

Рис. 3. Додавання двох матриць

4.4. Множення матриць (3×4)×(4×2).

```
Оберіть дію:
1. Транспонування матриці
2. Множення матриці на скаляр
3. Додавання матриць
4. Віднімання матриць
5. Множення матриць
5
Введіть кількість рядків та стовпчиків матриці номер 1 (через пробіл):
3 4
Введіть саму матрицю номер 1:
1.5 -2 0.5 3.2
4.1 0 -1.5 2.5
-0.5 3.4 2.0 -1
Введіть кількість рядків та стовпчиків матриці номер 2 (через пробіл):
4 2
Введіть саму матрицю номер 2:
2.0 -1.5
0.5 4
-3 1.2
1 0.5
Результат:
    3.70    -8.05
   15.20    -6.70
   -6.30    16.25
```

Рис. 4. Множення матриць

4.5. Обробка помилок введення та несумісних розмірів.

```
Оберіть дію:
1. Транспонування матриці
2. Множення матриці на скаляр
3. Додавання матриць
4. Віднімання матриць
5. Множення матриць
3
Введіть кількість рядків та стовпчиків матриці номер 1 (через пробіл):
ф 2

ПОМИЛКАААААА!!!введіть число ще раз : 2 2
Введіть саму матрицю номер 1:
2 1
3 2
Введіть кількість рядків та стовпчиків матриці номер 2 (через пробіл):
2 5
Помилка: розміри матриць повинні бути однаковими для додавання!
```

Рис. 5. Помилка вводу та несумісні розміри при додаванні

```
Оберіть дію:
1. Транспонування матриці
2. Множення матриці на скаляр
3. Додавання матриць
4. Віднімання матриць
5. Множення матриць
5
Введіть кількість рядків та стовпчиків матриці номер 1 (через пробіл):
3 3
Введіть саму матрицю номер 1:
3 2 1
3 3 2
4 4 4
Введіть кількість рядків та стовпчиків матриці номер 2 (через пробіл):
5 5
Помилка: кількість стовпчиків першої матриці має дорівнювати кількості рядків другої!
```

Рис. 6. Помилка розмірів при множенні матриць

5. Першокод програми.

```
#include <stdio.h>
#include <math.h>
#include <ctype.h>
#include <stdbool.h>
#include <time.h>
#include <stdlib.h>

void i_input(int *ptr){
     if(scanf("%d",ptr)!=1){
        while (getchar() != '\n');
        printf("\nПОМИЛКАААААА!!!введіть число ще раз : ");
        i_input(ptr);
        return;
    }
    return;
}

void f_input(double *ptr){
     if(scanf("%lf",ptr)!=1){
        while (getchar() != '\n');
        printf("\nПОМИЛКАААААА!!!введіть число ще раз : ");
        f_input(ptr);
        return;
```

```
    }
    return;
}

double** allocmatrix(int r,int c){
    double **matr= (double**)malloc(r*sizeof(double *));
    for(int i =0;i< r;i++){
        matr[i]= (double *)malloc(c *sizeof(double));
    }
    return matr;
}
void freematr(double **matr ,int r){
    for(int i=0;i<r;i++) free(matr[i]);
    free(matr);
}
double** transponate(double **matr,int r,int c){
    double **res=allocmatrix(c,r);{
        for(int i=0;i< r;i++){
            for(int j =0;j<c ;j++){
                res[j][i]=matr[i][j];
            }
        }
    }
    return res;
}
double** mult_by_scalar(double **matr,int r,int c,double x){
    double **res=allocmatrix(r,c);
    for(int i=0;i< r;i++){
        for(int j=0;j<c;j++){
            res[i][j]=matr[i][j]*x;
        }
    }
    return res;
}
void input_matrix(double **matr,int r,int c) {
    for (int i=0; i<r;i++) {
        for (int j=0; j<c; j++) {
            f_input(&matr[i][j]);
        }
    }
}
void print_matrix(double **matr, int r, int c) {
    printf("Результат:\n");
    for (int i = 0;i<r;i++) {
        for (int j=0; j<c;j++) {
            printf("%8.2lf ", matr[i][j]);
        }
        printf("\n");
    }
}
double** add_matrixes(double **A,double **B,int r,int c){
    double **res=allocmatrix(r,c);
    for(int i=0;i<r;i++){
        for(int j=0;j<c;j++) res[i][j]=A[i][j]+B[i][j];
    }
    return res;
}
double** minus_matrixes(double **A,double **B,int r,int c){
    double **res=allocmatrix(r,c);
    for(int i=0;i<r;i++){
        for(int j=0;j<c;j++) res[i][j]=A[i][j]-B[i][j];
    }
    return res;
}
double** mult_by_matrix(double **A,int ra,int ca,double **B,int rb,int cb){
    double **res=allocmatrix(ra,cb);
    for(int i=0;i<ra;i++){
        for(int j=0;j<cb;j++){
            res[i][j]=0.0;
            for(int k=0;k<ca;k++){
                res[i][j]+=A[i][k]*B[k][j];
            }
        }
    }
    return res;
}
int main() {
    int temp;
    printf("Оберіть дію:\n");
    printf("1. Транспонування матриці\n");
    printf("2. Множення матриці на скаляр\n");
    printf("3. Додавання матриць\n");
    printf("4. Віднімання матриць\n");
    printf("5. Множення матриць\n");
```

```
i_input(&temp);

int row,col,row2,col2;
double **matrix=NULL;
double **matrix2=NULL;
double **result=NULL;
switch (temp)
{
case 1:
    printf("Введіть кількість рядків та стовпчиків матриці (через пробіл):\n");
    i_input(&row);
    i_input(&col);
    printf("Введіть саму матрицю:\n");
    matrix = allocmatrix(row,col);
    input_matrix(matrix,row,col);

    result=transponate(matrix,row,col);
    print_matrix(result,col,row);

    freematr(matrix, row);
    freematr(result, col);
    break;

case 2:
    printf("Введіть кількість рядків та стовпчиків матриці (через пробіл):\n");
    i_input(&row);
    i_input(&col);
    printf("Введіть саму матрицю:\n");
    matrix = allocmatrix(row,col);
    input_matrix(matrix,row,col);

    double scalar;
    printf("Введіть скаляр: ");
    f_input(&scalar);

    result=mult_by_scalar(matrix,row,col,scalar);
    print_matrix(result,row,col);
    freematr(matrix, row);
    freematr(result, row);
    break;

case 3:
    printf("Введіть кількість рядків та стовпчиків матриці номер 1 (через пробіл):\n");
    i_input(&row);
    i_input(&col);
    printf("Введіть саму матрицю номер 1:\n");
    matrix=allocmatrix(row,col);
    input_matrix(matrix,row,col);

    printf("Введіть кількість рядків та стовпчиків матриці номер 2 (через пробіл):\n");
    i_input(&row2);
    i_input(&col2);
    if (row!= row2 || col!= col2) {
        printf("Помилка: розміри матриць повинні бути однаковими для додавання!\n");
        freematr(matrix,row);
        return 1;
    }
    printf("Введіть саму матрицю номер 2:\n");
    matrix2=allocmatrix(row2,col2);
    input_matrix(matrix2,row2,col2);

    result =add_matrixes(matrix, matrix2, row, col);
    print_matrix(result, row, col);

    freematr(matrix, row);
    freematr(matrix2, row2);
    freematr(result, row);
    break;

case 4:
    printf("Введіть кількість рядків та стовпчиків матриці номер 1 (через пробіл):\n");
    i_input(&row);
    i_input(&col);
    printf("Введіть саму матрицю номер 1:\n");
    matrix=allocmatrix(row,col);
    input_matrix(matrix,row,col);

    printf("Введіть кількість рядків та стовпчиків матриці номер 2 (через пробіл):\n");
    i_input(&row2);
    i_input(&col2);
    if (row!= row2 || col!= col2) {
        printf("Помилка: розміри матриць повинні бути однаковими для віднімання!\n");
        freematr(matrix,row);
        return 1;
```

```
        }

        printf("Введіть саму матрицю номер 2:\n");
        matrix2=allocmatrix(row2,col2);
        input_matrix(matrix2,row2,col2);

        result =minus_matrixes(matrix, matrix2, row, col);
        print_matrix(result, row, col);

        freematr(matrix, row);
        freematr(matrix2, row2);
        freematr(result, row);
        break;

    case 5:
        printf("Введіть кількість рядків та стовпчиків матриці номер 1 (через пробіл):\n");
        i_input(&row);
        i_input(&col);
        printf("Введіть саму матрицю номер 1:\n");
        matrix=allocmatrix(row,col);
        input_matrix(matrix,row,col);

        printf("Введіть кількість рядків та стовпчиків матриці номер 2 (через пробіл):\n");
        i_input(&row2);
        i_input(&col2);
        if (col!=row2) {
            printf("Помилка: кількість стовпчиків першої матриці має дорівнювати кількості рядків другої!\n");
            freematr(matrix,row);
            return 1;
        }

        printf("Введіть саму матрицю номер 2:\n");
        matrix2=allocmatrix(row2,col2);
        input_matrix(matrix2,row2,col2);

        result =mult_by_matrix(matrix,row,col ,matrix2,row2,col2);
        print_matrix(result, row,col2);

        freematr(matrix,row);
        freematr(matrix2,row2);
        freematr(result,row);
        break;

    default:
        printf("Такої функції не існує!");
        break;
    }
}
```